FIT5226 Project

Stage 1 - Tabular Q-Learning

This document describes Phase 1 of the FIT 5226 project, which will be conducted in two stages. These have been described during the Seminar in Week 1 and are briefly summarized below. The first stage is the foundation for the second one.

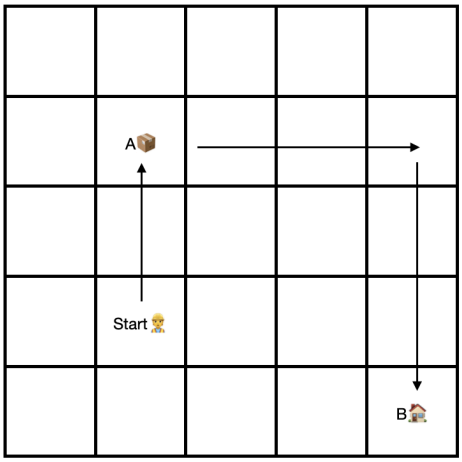
Assessment	Topic	Release	Due	Weight	Assessment mode
in-semester	Project Stage 1 (Table-based RL, single agent)	Wk3	April 11	15%	Individual
in-semester	Project Stage 2 (MARL coordination task)	Wk 6	Wk 9	35%	Individual
exam	All topics but no programming			50%	

This project constitutes your *entire* in-semester assessment for the unit..

Phase 1 (the current Phase) concerns *Tabular Q-Learning for Single Agents*.

Tasks for Stage 1

You will write code for a single agent in a square grid world of size *n* to learn a simple transport task. The agent's task is to pick up the item at location A and deliver it to a fixed target location B. A is not known in advance, ie. it varies each time the agent needs to solve the task. B is the bottom right corner of the grid at coordinates (*n*,*n*). The coordinates of A are part of the state information (observation) that the agent receives.



Your agent has 8 actions that it can execute, namely to step to any neighboring field (including along diagonals). It starts at a random location. When it reaches location A it *automatically* picks up the item, when it reaches location B it *automatically* discharges the item. No specific action is required from the agent for picking up or dropping an item, it only needs to step onto the corresponding field. At this point, it has completed its task.

The agent is allowed to observe its own location, the location of A and whether it carries an item.

The task the agent has to learn is to pick up the load at A and deliver it to B taking as few steps as possible regardless of its (random) starting position.

1. **Reward Structure:** Design and describe a reward structure that will allow the agent to learn this task efficiently using reinforcement learning. Document this in your notebook.
2. **Environment:** Implement a grid world, in which the agent can move and execute its task. To make the task manageable, we use a 5x5 grid world. However, your code should be set up to work for any size (so use parameters for the size and for the target location). We simply choose a smaller grid here to limit memory and time requirements for the training. Note that you will have to integrate this with a visualization in the final task. It is highly advisable that you consider this for your code design right from the beginning.
3. **Learning:** Implement a table-based Q-Learning algorithm for this agent in Python as a Jupyter notebook. You are not allowed to use any reinforcement learning libraries for this, your Q-learning must be implemented "from scratch". You are, of course, allowed to use all modules in the standard distribution of Python (e.g. random). The only library you should need beyond this is Numpy (and later Matplotlib for the visualization). If you want to use any other additional libraries apart from Numpy and Matplotlib check with your tutor beforehand whether these are admissible.

4. **Training and Testing:** Train your Q-Learner. Devise a test procedure and metrics that you can use to show that your agent learns the task successfully and that it learns to solve the task independently of the location of A.
5. **Documentation:** Use the metrics you defined above to document in writing that your agent learns its task. Clearly explain how your test procedure and metrics work. You will also have to be ready to demonstrate this to your tutor and to fully explain this in an interview. Make sure to document all code appropriately. Do so within your notebook.
6. **Visualisation:** Design, implement appropriate visualisations to show that your agent learns the task successfully. This may but does not have to include animations. You can also rely on plots and other forms of visualisation. Provide these visualisations together with appropriate explanations in the notebook. If you use animations, you may use the grid-world code from the first tutorial to visualise that and how your agent learns the task. For example, use the visualisation to show typical runs at different stages of the training. You are, of course, free to write your own visualisation code. The code from Laboratory 1 is provided only to make your task easier.

Performance Level and Training Budget

To achieve full marks, your agent must learn to solve 100% of the possible scenarios in 10 steps or less within less than 20,000 training episodes.

Submission Instructions

Submission will be via the Moodle platform. Detailed submission instructions will be published on Moodle in the Assignment section.

Use of Generative AI

You are allowed to use Generative AI to solve your assignment. If you decide to do so, you must treat the AI like another external author (as a non-authoritative author whom you mistrust, given how much content is made up by Chat GPT and similar AIs). It is entirely your own responsibility that the content is correct and you can only use generated content to the extent that you could use materials provided by an external author. The AI is not part of your project team. If you use code that an AI produced you must be able to fully explain every detail of that code.

You must give a declaration that fully explains how and for which components you used generative AI.

Any use of generative AI must be appropriately acknowledged (see Learn HQ).